

Jared Medlin
WGU D213 Task 2
3/12/2026

A.

1.

I will be using the Yelp dataset.

Research Question: For a restaurant chain using Yelp reviews as a voice-of-customer channel, can a neural NLP model accurately classify sentence-level sentiment?

2.

Goals:

- Build an automated sentence-level sentiment classifier for Yelp restaurant feedback.
- Evaluate whether the model is accurate enough for operational monitoring.
- Ensure the model is reliable and not biased toward one class.
- Produce deployable outputs an organization can act on.

3. The type of neural network used is a feedforward neural network with a learned word embedding layer. The architecture consists of an embedding layer, a GlobalAveragePooling1D layer, and two dense layers. This type of network is good for binary classification of text since the embedding layer learns the numerical representation of words during training, then the pooling layer converts the sequences into representational size that the dense layers can then classify (Keras, n.d.).

B.

1.

4 rows of non ASCII characters and 4 rows of unusual characters were found. They appear to be the same rows, so just 4 issues. See attached code for more detail:

```
#Other unusual characters
common = set(string.ascii_letters + string.digits + string.punctuation + " \t\n\r")
def unusual_chars(s: str):
    return sorted(set(ch for ch in s if ch not in common))

unusual = df["text"].apply(unusual_chars)
unusual_mask = unusual.apply(lambda xs: len(xs) > 0)

#Printing the culprits
print("Rows with unusual chars:", unusual_mask.sum())
print(unusual[unusual_mask].head(10).to_list())
print(df.loc[unusual_mask, "text"].head(10).to_list())
```

Non-ASCII rows: 4

['My fiancé and I came in the middle of the day and we were greeted and seated right away.', 'I really enjoyed Crema Café before they expanded; I even told friends they had the BEST breakfast.', 'The crêpe was delicate and thin and moist.', "The only thing I wasn't too crazy about was their guacamole as I don't like it puréed."]

Emoji rows: 0

[]

Rows with unusual chars: 4

[['é'], ['é'], ['è'], ['é']]

['My fiancé and I came in the middle of the day and we were greeted and seated right away.', 'I really enjoyed Crema Café before they expanded; I even told friends they had the BEST breakfast.', 'The crêpe was delicate and thin and moist.', "The only thing I wasn't too crazy about was their guacamole as I don't like it puréed."]

Since modern tokenizers (should) handle accents fine, these can be left as-is.

Vocabulary size:

```
#Vocabulary size
import re

token_pattern = re.compile(r"[A-Za-zÀ-ÿ]+") # keeps contractions + accented latin chars

tokens = df["text"].str.lower().apply(lambda s: token_pattern.findall(s))

vocab = set()
for toks in tokens:
    vocab.update(toks)

print("Vocabulary size (word-level):", len(vocab))
print("Sample vocab:", sorted(list(vocab))[:30])
```

Vocabulary size (word-level): 2042

Sample vocab: ['a', 'about', 'above', 'absolute', 'absolutely', 'absolutley', 'accident', 'accommodations', 'accommodate', 'accordingly', 'accountant', 'ache', 'acknowledged', 'across', 'actual', 'actually', 'added', 'affordable', 'after', 'afternoon', 'again', 'ago', 'ahead', 'airline', 'airport', 'ala', 'albondigas', 'all', 'allergy', 'almonds']

Vocabulary size was 2042 unique word tokens.

Since the dataset is small, I don't want to overfit the data. I decided on an embedding length of 64, which would be 204200 parameters.

Sequence length (see attached code for more detail):

```

count      1000.000000
mean       10.830000
std        6.213832
min        1.000000
25%        6.000000
50%       10.000000
75%       15.000000
max       32.000000
Name: text, dtype: float64
p90: 20.0
p95: 23.0
p97.5: 25.0
p99: 28.0
p100: 32.0
Max_len=16 covers 80.7% of sentences
Max_len=20 covers 90.4% of sentences
Max_len=23 covers 96.0% of sentences
Max_len=25 covers 97.8% of sentences
Max_len=28 covers 99.4% of sentences
Max_len=32 covers 100.0% of sentences

```

Mean length is 10.83

Max length is 32.

I'm going with 25 since it has a coverage of over 97% and is efficient compared to higher values.

2.

Tokenization Process

Goals:

- Convert raw text into model-readable sequences
- Reserve sentiment-carrying information
- Standardize surface forms to reduce noise and vocabulary explosion
- Create a consistent preprocessing pipeline

(Ali Awan)

I used tensorflow 2.15 for this process. This was the newest stable version I could get to run.

3.

Padding is the technique used to manage the spatial dimensions of input data so they are the same, as required by matrices (DeepAI, n.d.). For my model, I used the command `output_sequence_length=Max_LEN` where the max length is 25. After that Keras handles all the padding automatically. Padding is applied after the text sequence.

```
vectorizer.adapt(X_train.astype(str).to_numpy())
```

```
vocab = vectorizer.get_vocabulary()
print("Vocabulary size:", len(vocab))
print("First 20 tokens:", vocab[:20])
```

```
WARNING:tensorflow:From C:\Users\weepo\pythonlab\Lib\site-packages\keras\src\utils\tf_utils.py:492:
The name tf.nn.conv2d is deprecated. Please use tf.nn.conv2d_v2 instead.
```

```
Vocabulary size: 1673
```

```
First 20 tokens: ['', '[UNK]', 'the', 'and', 'i', 'was', 'a', 'to', 'is', 'this', 'it', 'of', 'food',
, 'not', 'in', 'place', 'for', 'good', 'service', 'my']
```

```
sample = tf.constant(["The food isn't good, not coming back."])
```

```
print("Standardized:")
print(custom_standardize(sample).numpy()[0].decode())
```

```
print("Token IDs:")
print(vectorizer(sample).numpy())
```

```
Standardized:
```

```
the food isn't good not coming back
```

```
Token IDs:
```

```
[[ 2 12 393 17 13 145 30 0 0 0 0 0 0 0 0 0 0]
 [ 0 0 0 0 0 0 0 0]]
```

4.

I have two categories of sentiment, just negative (0) and positive (1). The final dense layer uses sigmoid activation, which gives a probability between 0 and 1 for binary classification.

5.

The raw review data was loaded and split into a text column and a binary label column, with 0 being negative and 1 being positive. The data was then divided into 700 train, 150 validation, and 150 test splits (based on industry standard). Stratification was used to preserve the 50/50 positive/negative balance across all three splits. The TextVectorization layer was fit on the training set to convert raw text into integer token sequences. All sequences were padded or truncated to the limit of 25 tokens, and then data was batched into groups of 32 for efficient training.

6.

See data attached.

C.

1.

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 64)	107072
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense (Dense)	(None, 64)	4160
dense_1 (Dense)	(None, 1)	65

```

Total params: 111297 (434.75 KB)
Trainable params: 111297 (434.75 KB)
Non-trainable params: 0 (0.00 Byte)

```

2.

There are four layers: Embedding, GlobalAveragePooling1D, Dense 64 hidden, and Dense output.

Embedding is the heaviest layer with 107,072 parameters. It acts as a lookup table that converts integer token IDs into a vector of 64 numbers.

GlobalAveragePooling1D has no parameters. It just takes the sequence of 25 token vectors and averages them into a single vector of 64 numbers.

Dense 64 hidden has 4160 parameters. It is a fully connected hidden layer that allows the model to learn non-linearly, and thus learn more complex patterns than straight linear transformation.

Dense output has 65 parameters. It takes the 64 outputs from the previous layer and turns them into a probability score between 0 and 1, with 1 being positive and 0 being negative.

3.

Justification of Hyperparameters:

Activation functions:

ReLU (Rectified Linear Unit) outputs 0 for any negative input and the raw value for any positive input. It is standard for hidden layers since it is computationally cheap and works well in practice generally.

Sigmoid gives the output by squishing any inputs into a value between 0 and 1, which is interpreted as a probabilistic weight between a negative vs a positive sentiment. It is perfect for this task since it is a binary task.

Nodes Per Layer:

Embedding layer has 64. It is a good number for smaller datasets since it is large enough to capture meaningful semantic relationships between words, but small enough to not overload the training set.

Dense hidden layer has 64 nodes. Since the input from the previous layer is already 64 wide it avoids any potential bottlenecks.

Output layer has 1 node. It is a binary problem, so we only need 1 output.

Loss Function:

Binary crossentropy is the standard for binary classification, so that's what was used.

Optimizer:

Adaptive Moment Estimation (adam) is well-suited for NLP tasks and is the default choice for most neural network projects, so this was used.

Stopping Criteria:

Early Stopping will stop the learning of the model once the `val_loss` value indicates that the model is no longer improving. This ensures the model does not overfit.

Evaluation metric:

Accuracy, or the percentage of correctly classified reviews, is the metric used to determine efficacy of the model. This is because the dataset is perfectly split into 500 positive and 500 negative reviews, so accuracy is a straightforward metric to interpret and report.

(IBM, n.d.)

D.

1.

The model was configured to run up to 50 epochs and used early stopping by monitoring `val_loss` with a patience of 3 and `restore_best_weights=True` to ensure there is no overfitting.

As shown in the screenshot, the training was halted at 12 epochs, meaning that the model's `val_loss` stopped improving around epoch 9 and rose for 3 epochs, so the model's weights were restored to epoch 9. This is important because without the early stopping, the model would severely overfit if left to train further up to 50 epochs. Early stopping just automates when to stop without trial and error.

```
#Early stopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    X_train_vec, y_train,
    validation_data=(X_val_vec, y_val),
    epochs=50,
    batch_size=32,
    callbacks=[early_stop]
)
```

Epoch 1/50
WARNING:tensorflow:From C:\Users\weepo\pythonlab\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_agerly_outside_functions is deprecated.

```
22/22 [=====] - 1s 9ms/step - loss: 0.6929 - accuracy: 0.5286 - val_loss: 0.6905 - val_accuracy: 0.5267
Epoch 2/50
22/22 [=====] - 0s 3ms/step - loss: 0.6857 - accuracy: 0.5971 - val_loss: 0.6852 - val_accuracy: 0.6333
Epoch 3/50
22/22 [=====] - 0s 3ms/step - loss: 0.6693 - accuracy: 0.7857 - val_loss: 0.6721 - val_accuracy: 0.6733
Epoch 4/50
22/22 [=====] - 0s 3ms/step - loss: 0.6297 - accuracy: 0.8243 - val_loss: 0.6485 - val_accuracy: 0.7267
Epoch 5/50
22/22 [=====] - 0s 3ms/step - loss: 0.5584 - accuracy: 0.8743 - val_loss: 0.6070 - val_accuracy: 0.7000
Epoch 6/50
22/22 [=====] - 0s 3ms/step - loss: 0.4535 - accuracy: 0.9086 - val_loss: 0.5615 - val_accuracy: 0.7200
Epoch 7/50
22/22 [=====] - 0s 3ms/step - loss: 0.3491 - accuracy: 0.9286 - val_loss: 0.5247 - val_accuracy: 0.7400
Epoch 8/50
22/22 [=====] - 0s 3ms/step - loss: 0.2554 - accuracy: 0.9543 - val_loss: 0.5165 - val_accuracy: 0.7400
Epoch 9/50
22/22 [=====] - 0s 3ms/step - loss: 0.1933 - accuracy: 0.9671 - val_loss: 0.4946 - val_accuracy: 0.7467
Epoch 10/50
22/22 [=====] - 0s 2ms/step - loss: 0.1421 - accuracy: 0.9800 - val_loss: 0.4979 - val_accuracy: 0.7533
Epoch 11/50
22/22 [=====] - 0s 2ms/step - loss: 0.1083 - accuracy: 0.9871 - val_loss: 0.4983 - val_accuracy: 0.7467
Epoch 12/50
22/22 [=====] - 0s 3ms/step - loss: 0.0848 - accuracy: 0.9900 - val_loss: 0.5158 - val_accuracy: 0.7733
```

2.

The model shows signs of overfitting. The training accuracy reached 98% while validation accuracy stopped at around 75%, which is a large gap. The divergence between training and validation loss is a sign of overfitting. This likely occurred due to the dataset's size. With only 700 training samples and almost all of the 111,000 parameters in the embedding layer, the model has more capacity than the data requires, making it prone to overfitting. The main action taken to prevent overfitting was the early stopping and restoring back to the best weights at epoch 9.

3.

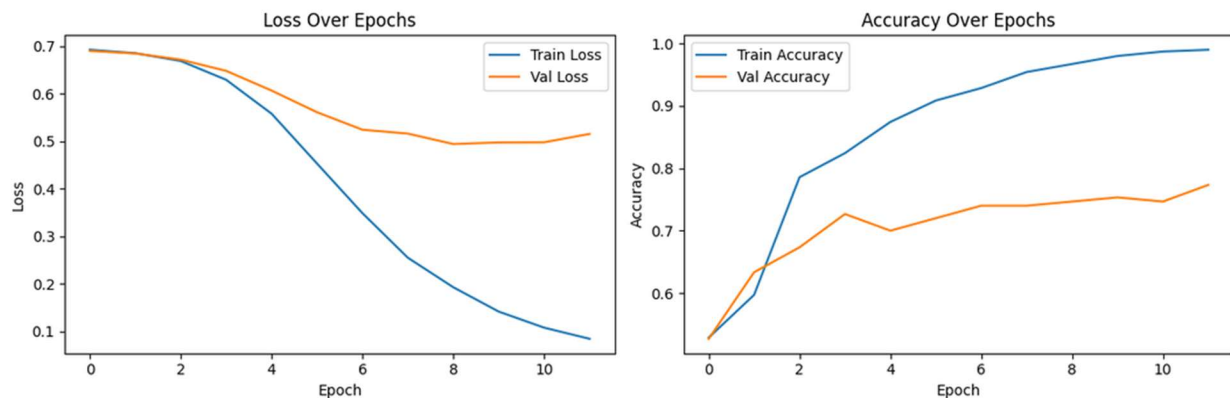
```
#Training visualizations
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_title('Loss Over Epochs')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()

axes[1].plot(history.history['accuracy'], label='Train Accuracy')
axes[1].plot(history.history['val_accuracy'], label='Val Accuracy')
axes[1].set_title('Accuracy Over Epochs')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()
```



(from cell 20)

4.

```
#Model evaluation
#Evaluate on test set
loss, acc = model.evaluate(X_test_vec, y_test)
print(f"Test Loss: {loss:.4f} | Test Accuracy: {acc:.4f}")

5/5 [=====] - 0s 1ms/step - loss: 0.3749 - accuracy: 0.8333
Test Loss: 0.3749 | Test Accuracy: 0.8333
```

Model had an accuracy of 83.33%. Given that the model is based on a dataset containing 50/50 data, the accuracy value being this high means that the model is functioning well; it correctly classifies at least 4 out of every 5 reviews. It is important to note, though, that almost 1 out of every 5 reviews is classified incorrectly, which is pretty high for any practical use. The gap between training accuracy of 98% and testing accuracy of 83.33% means that the model is constrained by overfitting. A larger dataset would solve this.

E.

```
model.save("sentiment_model.keras")
print("Model saved.")
```

Model saved.

See attached code.

F.

The created neural network takes a raw Yelp review as a text input, converts it into numerical representations indicating positive and negative sentiment, and then gives a single probability score on the review's overall positive sentiment. The architecture choice of GlobalAveragePooling plus simple embedding made for a very fast to train and easy to interpret model. However, the model loses some nuance. For instance, it doesn't take into account things like word order or context, since it dissolves all words into a single representation. The addition of a single word into a sentence could flip the meaning entirely normally, but to this model they would be nearly identical.

Overall, the model is well-suited to the project given its scope despite its drawbacks and limitations. It gives an accuracy that is high enough to be useful, if imperfect, at 83.33%. If you're looking for overall sentiment of a large number of reviews, this model will work perfectly fine.

G.

Recommendations:

The model is capable of being deployed for general use for a business like a restaurant chain. It will analyze a large number of the incoming reviews and give a general sentiment that the reviews are giving almost instantly. If the number of negative reviews is too high, further manual reading and investigation is advised, as the model cannot give context.

In the future, for improved modeling it is advised to get a larger dataset, since this model became overfitted so quickly. A larger dataset will allow the model to train for higher accuracy and be better-suited for industry use.

H.

Attached

I-J.

Sources:

Ali Awan <https://www.datacamp.com/blog/what-is-tokenization>

DeepAI. (n.d.). *Padding (Machine learning) definition*. <https://deepai.org/machine-learning-glossary-and-terms/padding>

Keras. (n.d.). *GlobalAveragePooling1D layer*.
https://keras.io/api/layers/pooling_layers/global_average_pooling1d/

IBM. (n.d.). *What is hyperparameter tuning?*
<https://www.ibm.com/think/topics/hyperparameter-tuning>

K.

Thank you for reading! 😊